Message Token Modifiers & Audio Parsing Specification

Two-Stage Pipeline Architecture, Formatting Modifiers, and Playback Directives

Firmware: ESP32-S2 (ckt-defect) | **Source:** messages.cpp, parser.cpp | **Test Suite:** test/messages

1. System Architecture & Dual-Mode Execution Model

Message generation in the defect detector firmware operates as a coordinated two-stage pipeline. **Stage 1** (transformMessage) expands variable placeholders into concrete text and applies column formatting. **Stage 2** (parseTask) runs as a dedicated FreeRTOS task, converting the expanded token stream into real-time spoken audio and synthetic waveforms.

- **Display Mode (breakDigits = false):** Formats text for the 20x4 LCD screen. Preserves numbers as contiguous values (e.g. "124"), enforces exact field padding/alignment constraints, and converts single backslashes (\) into newlines (\n) to split rows.
- **Spoken Mode (breakDigits = true):** Prepares text for the voice synthesizer. Splits numbers into individual whitespace-separated digits and tokens (e.g. "1 2 4", "3 4 6 . 9") so the audio engine can perform word-by-word vocabulary lookups. Integer padding constraints are bypassed.
- **Pass-Through Directives:** Audio generator tokens (#tone, #pause, #count, #ord) are untouched by Stage 1 and executed dynamically by Stage 2.

2. Stage 1: transformMessage Reference

Syntax: #<baseToken>[:[±]N[.M]] — : separates the modifier. ± selects alignment (omitted/positive = right-aligned / left space padded; - = left-aligned / right space padded). N is field width; .M specifies fractional digits (for #milepost).

Token	Modifier Syntax	Value Source & Type	Display Mode (breakDigits=false)	Spoken Mode (breakDigits=true)
****	None	Control Character	Converted to newline (\n) for LCD lines	Converted to newline (\n)
****	None	Escape Sequence	Converted to literal backslash (\)	Converted to literal backslash (\)
#milepost	`:[±]N[.M]` *(def: 1.1)*	cfg.milepost (tenths of a mile, int)	`#milepost:4.1 with 3469 $\to$ " 346.9"` `#milepost:-4.1 $\to$ "346.9 "`	`Padding ignored; digits split: "3 4 6 . 9"`
#track	`:[±]N` *(optional)*	cfg.trackName[trackNum] (string)	Truncates to |N| if longer; pads left (N>0) or right (N<0). #track:4 $\to$ "Main"	Truncates or pads string field verbatim
#direction	`:[±]N` *(optional)*	data.direction (1=dir1, 2=dir2, other=empty)	Truncates or pads direction name. Returns empty string "" if direction is unknown	Truncates or pads direction string
#rail	`:[±]N` *(optional)*	cfg.railName (e.g. "North/South") & data.rail	Splits on '/': odd rail selects 1st half ("North"), even selects 2nd ("South"). Truncates/pads.	Selects rail side; truncates/pads
#axle	`:[±]N` *(def: 1)*	data.axleCount (defect location, int)	`#axle:3 with 4 $\to$ " 4" (right-aligned)` `#axle:-3 $\to$ "4 " (left-aligned)`	`Emitted as "4"`
#axles	`:[±]N` *(def: 1)*	data.totalAxles (total axles, int)	`#axles:5 with 124 $\to$ " 124"` `#axles:-5 $\to$ "124 "`	`Padding ignored; digits split: "1 2 4"`
#speed	`:[±]N` *(def: 1)*	data.speed (train speed mph, int)	`#speed:3 with 45 $\to$ " 45"` `#speed:-3 $\to$ "45 "`	`Padding ignored; digits split: "4 5"`
#temp	`:[±]N` *(def: 1)*	TemperatureManager (ambient temp + 0.5, int)	`#temp:3 with 72 $\to$ " 72"`	`Padding ignored; digits split: "7 2"`
#defectlist	None	data.defects (vector of pre-transformed strings)	Concatenates all defect details with single space separator: e.g. "HOT_BOX DRAG_EQ"	Concatenates detail messages with single space separator

3. Stage 2: parseTask Audio Directives & Speech Synthesis

parseTask executes on a dedicated FreeRTOS task, popping ParserObject items from parserQueue. It updates the LCD display message (protected by displayMutex) and evaluates the spoken message token by token (whitespace delimited).

Directive / Token	Syntax & Defaults	Synthesis Mechanism & Mathematical Formula	Examples & Behavior

#tone
Tone Generator
	`#tone`
	`#tone=M`
	`#tone=M,N`
	`(def: M=10, N=1)`
		Generates synthetic 1,000 Hz sine wave tone in RAM (ToneSound) at 16 kHz.
		• **M:** Duration in deciseconds (10 = 1.0 s; samples = $1600 \times M$).
		• **N:** Volume attenuation exponent: $\text{Amplitude} = 1 / 2^N$.
		• $N=0 \to 100\%$ (0 dB) • $N=1 \to 50\%$ (-6 dB)
		• $N=2 \to 25\%$ (-12 dB) • $N=3 \to 12.5\%$ (-18 dB)
			`• #tone $\to$ 1.0 s tone at 50% vol`
			`• #tone=5 $\to$ 0.5 s tone at 50% vol`
			`• #tone=10,0 $\to$ 1.0 s tone at 100% vol`
			`• #tone=20,2 $\to$ 2.0 s tone at 25% vol`

#pause
Silence Generator
	`#pause`
	`#pause=M`
	`(def: M=5)`
		Generates pure digital silence (SilenceSound) at 16 kHz.
		• **M:** Duration in deciseconds (samples = $1600 \times M$). Inserts natural acoustic pacing between phrases.
			`• #pause $\to$ 0.5 s pause (5 decisecs)`
			`• #pause=1 $\to$ 0.1 s brief pause`
			`• #pause=10 $\to$ 1.0 s pause`
			`• #pause=20 $\to$ 2.0 s pause`

#count **Numeric Counter**	`#count` *`(starts at 1)`*	Queries vocabulary word for current number: vocabGetWord(std::to_string(count)). Automatically increments count++ after queuing. Resets to 1 on each new message.	1st encounter speaks "1"; 2nd encounter speaks "2"; 3rd encounter speaks "3".
#ord **Ordinal Counter**	`#ord` *`(starts at 1)`*	Retrieves ordinal word: vocabGetWord(getOrdinalWord(count)). Supports range 1..9 ("first" .. "ninth"; >9 emits silence). Auto-increments count++. Shares counter state with #count.	Defect detail iteration: 1st $\to$ "first" 2nd $\to$ "second" 3rd $\to$ "third"
. and - **Punctuation**	`Single-char tokens`	When numbers are split by breakDigits=true, punctuation appears as isolated tokens. parseTask translates them before vocab lookup: • "." $\to$ "point" (looks up vocabGetWord("point")) • "-" $\to$ "minus" (looks up vocabGetWord("minus"))	• "3 4 6 . 9" $\to$ "three four six point nine" • "- 5" $\to$ "minus five"
Vocabulary & SFX Fallback	`Standard words`	Standard words and digits ("0"–"9") are loaded via vocabGetWord(token). **Diagnostic Fallback:** If vocabGetWord() returns NULL, parseTask plays the sound effect **"horn"** (PneumaticHornDouble3-16k.wav). This gives an unmistakable auditory alert during testing if a word is missing from vocabulary clips.	Missing word during bench test immediately triggers a loud dual train horn blast.

4. End-to-End Detector Message Workflows

Workflow 1: Real-Time Defect Alert (Hot Journal on Axle 48, Track 2 South Rail)

• **Raw Spoken:** "#tone #track hot journal Axle #axle #rail Rail" | **Display:** "#track\Hot Journal\Axle #axle #rail Rail"
• **Stage 1 Spoken (breakDigits=true):** "#tone track 2 hot journal axle 4 8 south rail"
• **Stage 2 Audio Playback:** [1.0s 1kHz Tone @ 50% vol] $\to$ "track" $\to$ "2" $\to$ "hot" $\to$ "journal" $\to$ "axle" $\to$ "4" $\to$ "8" $\to$ "south" $\to$ "rail"
• **Stage 1 LCD Display:** Line 0: Track 2 | Line 1: Hot Journal | Line 2: Axle 48 South Rail

Workflow 2: Clean Train Exit Summary (124 Axles, 45 mph, Milepost 346.9, 72°F)

• **Raw Spoken:** "... milepost #milepost #track #direction no defects repeat no defects total axles #axles train speed #speed temperature #temp degrees detector out"
• **Raw Display:** "#track\NO DEFECTS\AXLES:#axles:3 SPEED:#speed:3"
• **Stage 1 Spoken:** "... milepost 3 4 6 . 9 track 2 southbound no defects repeat no defects total axles 1 2 4 train speed 4 5 temperature 7 2 degrees detector out"
• **Stage 2 Audio:** . spoken as "point"; all digits ("3 4 6 9 1 2 4 4 5 7 2") spoken individually.
• **Stage 1 LCD Display:** Line 0: Track 2 | Line 1: NO DEFECTS | Line 2: AXLES:124 SPEED: 45

Workflow 3: Train Exit with Defect List Summary (Multiple Defects)

• **Detail Template:** "#ord hot journal Axle #axle #rail Rail" | **Exit Template:** "... you have a defect #defectlist total axles #axles detector out"
• **Pre-transformed Detail Messages in data.defects:**
Defect 1: "first hot journal axle 1 2 north rail" | Defect 2: "second dragging equipment near axle 8 4"
• **Expanded Exit Message:** "... you have a defect first hot journal axle 1 2 north rail second dragging equipment near axle 8 4 total axles 1 2 4 detector out"

5. Unit Test Suite Matrix (src/test/messages/test_messages.cpp)

Standalone verification using mock Arduino/FreeRTOS/NVS headers. All 27 tests pass with 100% coverage (./run).

#	Input Template	Expected Output	Mode	Tested Feature / Formatting Rule
1	`#milepost`	`3 4 6 . 9`	Spoken	Milepost spoken mode: splits digits and decimal point
2	`#milepost`	`346.9`	Display	Milepost display mode: compact representation
3	`#milepost:3.1`	`3 4 6 . 9`	Spoken	Spoken mode ignores integer width constraints
4	`#milepost:4.1`	`346.9`	Display	Right-aligned milepost, padded with 1 leading space
5	`#milepost:-4.1`	`346.9`	Display	Left-aligned milepost, padded with 1 trailing space
6	`#milepost:6.1`	`346.9`	Display	Right-aligned milepost, padded with 3 leading spaces
7	`#milepost:-6.1`	`346.9`	Display	Left-aligned milepost, padded with 3 trailing spaces
8	`#axle`	`4`	Display	Single-digit axle number
9	`#axle:3`	`4`	Display	Right-aligned axle number in 3-column field
10	`#axles:-3`	`1 2 4`	Spoken	Spoken total axles: negative width ignored, digits split
11	`#track`	`Main 1`	Display	Unmodified track name string
12	`#track:4`	`Main`	Display	Track string truncation (6 chars truncated to 4)
13	`#track:8`	`Main 1`	Display	Right-aligned track name padded to 8 columns
14	`#track:-8`	`Main 1`	Display	Left-aligned track name padded to 8 columns
15	`#direction`	`Northbound`	Display	Direction 1 lookup (data.direction = 1)
16	`#direction:4`	`Sout`	Display	Direction 2 truncation (data.direction = 2 truncated to 4)
17	`#direction:12`	`Northbound`	Display	Direction 1 right-aligned to 12 columns
18	`#direction:-12`	`Southbound`	Display	Direction 2 left-aligned to 12 columns
19	`#direction`		Display	Undefined direction (direction = 0) returns empty string
20	`#direction`		Display	Out-of-bounds direction (direction = 12) returns empty string
21	`#speed`	`4 5`	Spoken	Spoken speed: splits digits (45 -> "4 5")
22	`#temp`	`72`	Display	Display temperature from TemperatureManager
23	`#defectlist`	`HOT_BOX DRAG_EQ`	Display	Defect list concatenation with space separator
24	`milepost #milepost speed #speed:3 mph`	`milepost 346.9 speed 45 mph`	Display	Compound sentence with mixed tokens and modifiers
25	`Axles:#axles`	`Axles:124`	Display	Adjacent literal text and token
26	`Axles:#axles\Speed:#speed`	`Axles:124\nSpeed:45`	Display	Single backslash converted to LCD newline
27	`Axles:\\#axles`	`Axles:\124`	Display	Double backslash escaped to literal backslash

6. Quick Reference Summary Table

Directive	Stage	Syntax	Default	Mode Sensitivity	Operational Purpose
`\`	Stage 1	`\`	—	Both modes	LCD newline delimiter for row separation
`\\`	Stage 1	`\\`	—	Both modes	Escaped literal backslash character
`#milepost`	Stage 1	`:[±]N[.M]`	1.1	Digits split if spoken; padded if display	Milepost location (tenths of a mile)
`#track`	Stage 1	`:[±]N`	Verbatim	Truncates to |N|; pads left/right	Track identification name string
`#direction`	Stage 1	`:[±]N`	Verbatim	Truncates/pads; empty if unknown	Travel direction string (Northbound, etc.)
`#rail`	Stage 1	`:[±]N`	Verbatim	Slash partition on odd/even rail index	Defect rail side designation
`#axle, #axles`	Stage 1	`:[±]N`	1	Digits split if spoken; padded if display	Defect axle location / Total train axle count
`#speed, #temp`	Stage 1	`:[±]N`	1	Digits split if spoken; padded if display	Train speed (mph) / Ambient temperature
`#defectlist`	Stage 1	None	—	Space-separated list	Accumulated defect detail summary strings
`#tone`	Stage 2	`[=M[,N]]`	10,1	Audio waveform synthesis	1 kHz sine wave: M decisecs, $1/2^N$ volume
`#pause`	Stage 2	`[=M]`	5	Audio waveform synthesis	Digital silence: M decisecs duration
`#count, #ord`	Stage 2	None	1	Word lookup & auto-increment	Numeric ("1") / Ordinal ("first.."ninth") counter
`. , -`	Stage 2	`Single char`	—	Mapped to "point" / "minus"	Pronounces decimal point and negative signs
`<word>`	Stage 2	`String`	—	ROM/SD WAV clip lookup	Speaks vocabulary clip; falls back to horn SFX